Artificial Intelligence
CS-202

Topic: Machine Learning and Word Embeddings

Habiba Arshad

Department of Computer Science

University of Wah

Week-13 CLO's

This week lecture will cover the following CLO's:

- **CLO1:** Discuss the key concepts in the field of artificial intelligence
- **CLO 2: Implement** artificial intelligence techniques and case studies
- **CLO 3: Model** an automated solution for given problems using artificial intelligence paradigm

CLO's Mapping

CLO 1:

- Introduction to NLP and Machine Learning

CLO 2:

- Pre-processing and Feature Extraction

CLO 3:

- Machine Learning Models Implementation

NLP And Machine Learning

What is Machine Learning:

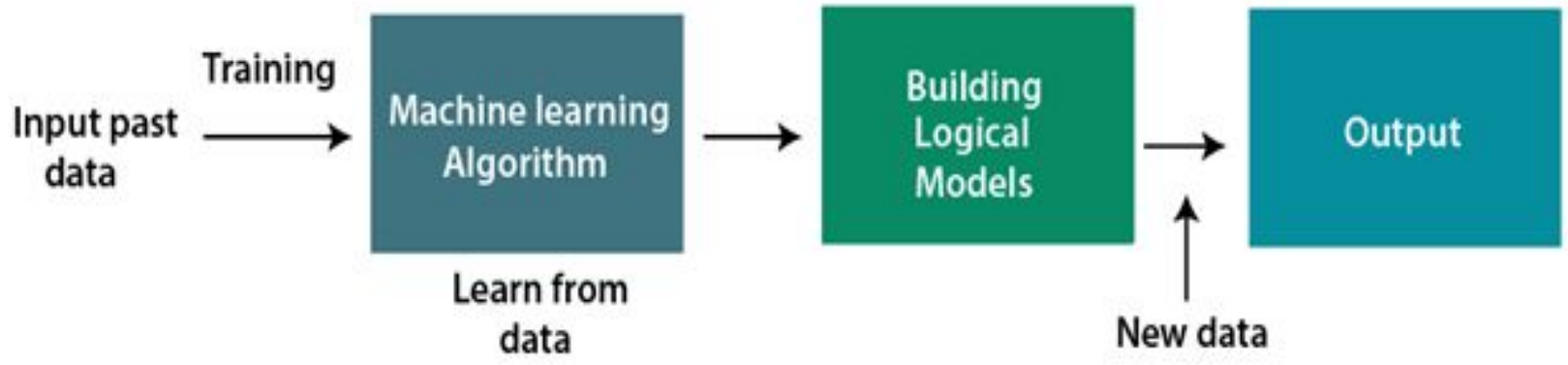
- “Machine learning is the field of study that enables a machine to automatically learn from data, improve performance from experiences, and predict things without being explicitly programmed.” (**Arthur Samuel 1959**)

What is Machine Learning

- Machine learning (ML) is a branch of artificial intelligence (AI) focused on enabling computers and machines to imitate the way that humans learn, to perform tasks autonomously, and to improve their performance and accuracy through experience and exposure to more data.

How does Machine Learning work

- A Machine Learning system **learns from historical data which in known as training data, builds the prediction models, and whenever it receives new data, predicts the output for it.**
- The accuracy of predicted output depends upon the amount of data, as the huge amount of data helps to build a better model which predicts the output more accurately.



Machine Learning Methods

Type	Description	Examples	Use Cases
1. Supervised Learning	Learn from labeled data to make predictions or classifications.	Linear Regression, Decision Trees, SVM	Spam detection, sentiment analysis, forecasting
2. Unsupervised Learning	Find patterns or groupings in unlabeled data.	K-Means, Hierarchical Clustering, PCA	Customer segmentation, topic modeling
3. Semi-Supervised Learning	Uses small labeled data + large unlabeled data for learning.	Modified supervised models	Medical imaging, speech recognition
4. Reinforcement Learning	Learn through trial and error to maximize rewards in an environment.	Q-Learning, Deep Q-Networks	Game AI, robotics, autonomous vehicles

Features of Machine Learning

- Machine learning uses data to detect various patterns in a given dataset.
- It can learn from past data and improve automatically.
- It is a data-driven technology.
- Machine learning is much similar to data mining as it also deals with the huge amount of the data.

Integrating NLP with Machine Learning

What is Integration?

- Using machine learning models to analyse and process natural language data.

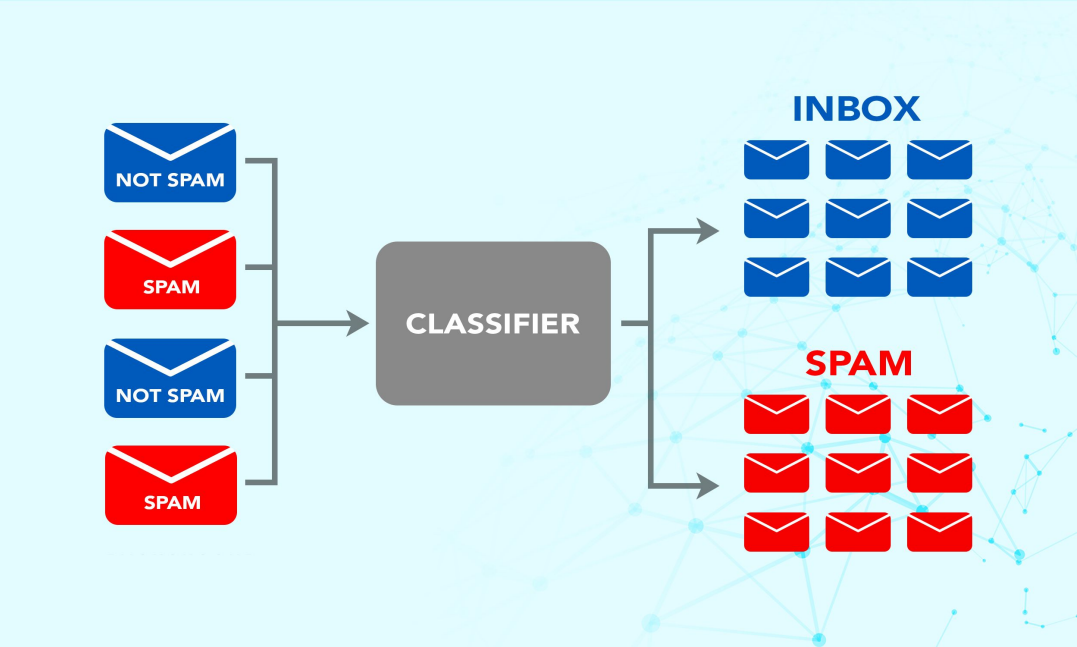
How It Works

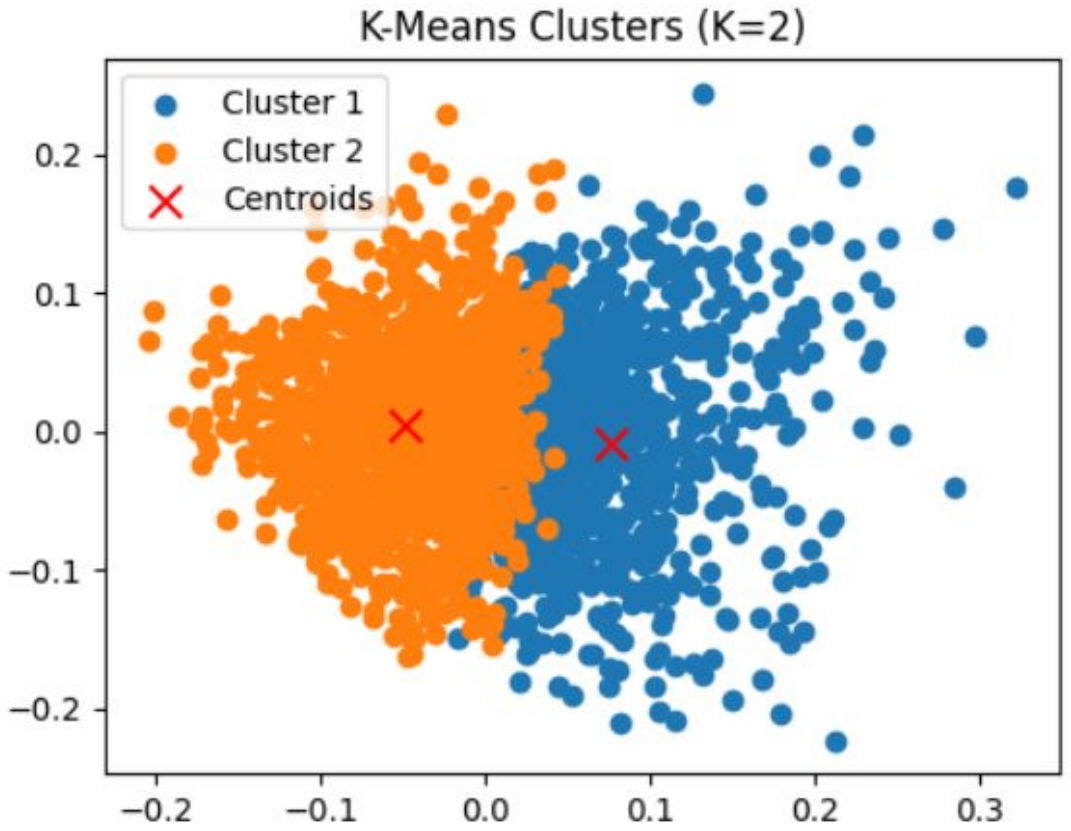
- **Data Collection**: Gathering raw text data.
- **Text Preprocessing**: Tokenization, removing stop words, stemming, etc.
- **Feature Extraction**: Converting text into numerical data (e.g., TF-IDF, Bag-of-Words, Word2Vec).
- **Model Application**: Applying ML models like Logistic Regression, Decision Trees, or Neural Networks.
- **Evaluation**: Assessing model performance (e.g., accuracy, F1-score).

Text Classification Techniques

- **Definition:**
  - Assigning predefined labels to text (e.g., Spam vs. Non-Spam emails).
- **Techniques:**
  - **Supervised Learning**:
    - Algorithms: Naive Bayes, SVM, Logistic Regression.
    - Example: Classify reviews as "Positive" or "Negative."
  - **Unsupervised Learning**:
    - Algorithms: K-Means, Hierarchical Clustering.
    - Example: Group documents into similar topics.
- **Real-World Example:**
  - Spam Email Detection:
    - Input: Email text.
    - Output: "Spam" or "Not Spam."

Text Classification





Word Embeddings

Word Embeddings

Word embedding in Natural Language Processing (NLP) is a technique used to represent words as numerical vectors in a continuous vector space.

These embeddings capture semantic meaning, syntactic properties, and relationships between words, allowing algorithms to process and analyze text more effectively.

Word Embeddings

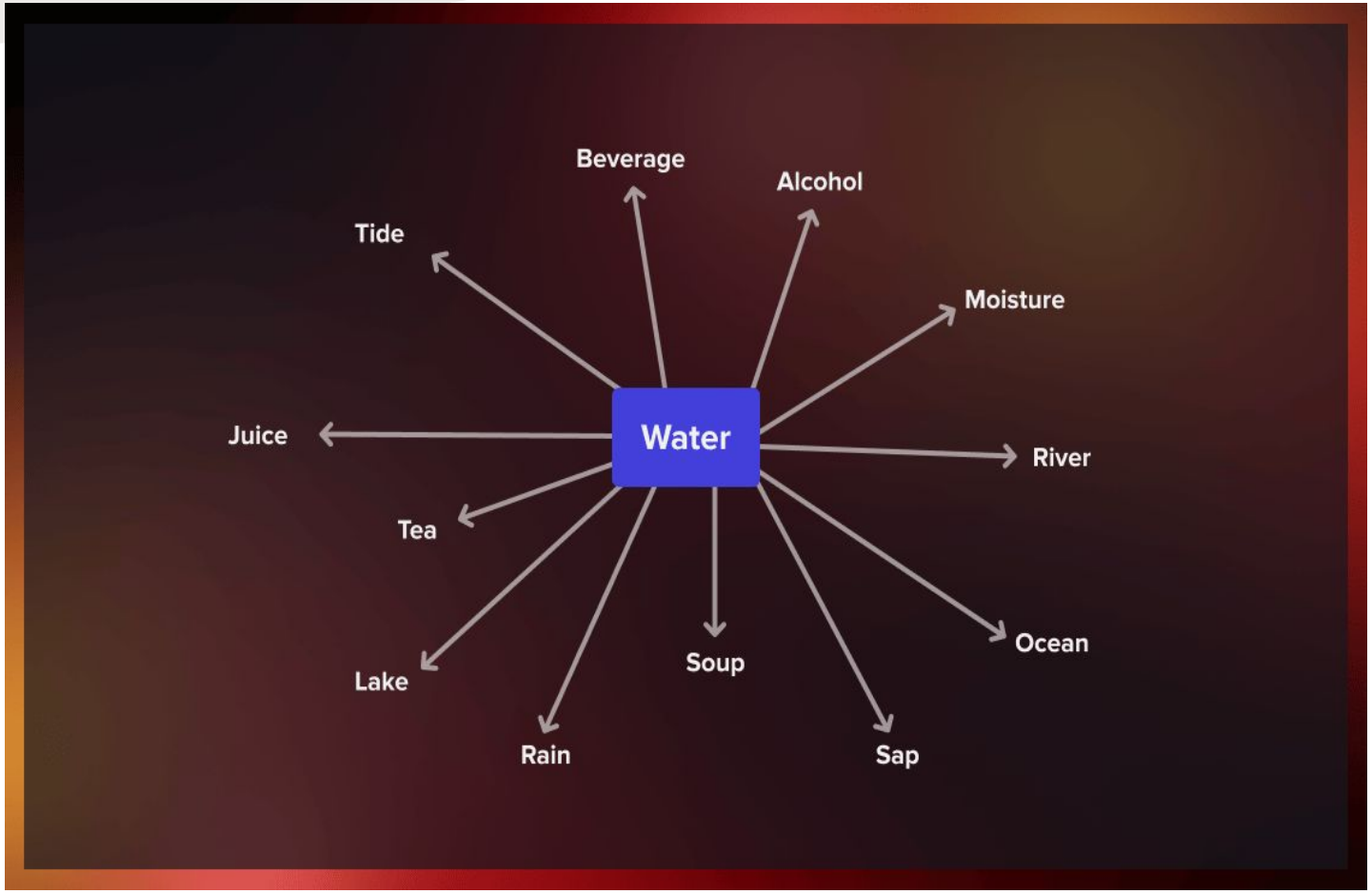
- For example, a computer can learn that “king” is more similar to “ruler” than “worker” by representing these words in a vector space where similar words are closer together.
- These embeddings are learned using language modeling and feature learning techniques, allowing computers to understand and process the semantic relationships between words.

Word Embeddings

The meaning of a term is determined by its context: the words that come before and after it, which is called the context window.

Typically, this window is four words wide, with four words to the left and right of the target term.

To create vector representations of words, we look at how often they appear together.



Techniques for Word Embeddings

- **TF-IDF**: Numerical representation based on frequency and importance.
- **Word2Vec**: Learns embeddings using neural networks
- Typically uses **100 to 300 dimensions**.
- **BERT:** Transformer-based contextual embeddings (each word's meaning depends on context).
- go much higher — up to **768 or even 1024 dimensions** per token.

Sample Code Example

```
from gensim.models import Word2Vec

# Example corpus
sentences = [["I", "love", "machine", "learning"], ["NLP", "is", "fun"]]

# Train Word2Vec model
model = Word2Vec(sentences, vector_size=100, window=5, min_count=1, workers=4)

# Get the embedding vector for the word 'machine'
vector = model.wv['machine']
print(vector)
```

- Values are floating-point numbers that encode the meaning of the word "machine" as learned by the model.
- Each word will have a different vector, and similar words will have similar vectors in terms of cosine similarity.

Word Embedding using TF-IDF

Analyze the collection of four documents as follows and determine the tf-idf scores for each term in each document.

*d*1 : "The sky is blue.

*d*2 : "The sun is bright today."

*d*3: "The sun in the sky is bright."

*d*4: "We can see the shining sun, the bright sun."

Word Embedding using TF-IDF

Step 1: Preprocess the documents

- Convert to lowercase and tokenize each document.
- Tokens:
  - d1: ["the", "sky", "is", "blue"]
  - d2: ["the", "sun", "is", "bright", "today"]
  - d3: ["the", "sun", "in", "the", "sky", "is", "bright"]
  - d4: ["we", "can", "see", "the", "shining", "sun", "the", "bright", "sun"]
- Remove punctuation and stopwords if necessary.

[the, sky, is, blue, sun, bright, today, in, we, can, see, shining]

Word Embedding using TF-IDF

Step 2: Calculate Term Frequency (TF)

- TF(t, d) = (Number of times term t appears in document d) / (Total number of terms in document d)
- TF for each document:

d1:

"the": 1/4 = 0.25
"sky": 1/4 = 0.25
"is": 1/4 = 0.25
"blue": 1/4 = 0.25

d2:

"the": 1/5 = 0.2
"sun": 1/5 = 0.2
"is": 1/5 = 0.2
"bright": 1/5 = 0.2
"today": 1/5 = 0.2

d3:

"the": 2/7 ≈ 0.286
"sun": 1/7 ≈ 0.143
"in": 1/7 ≈ 0.143
"sky": 1/7 ≈ 0.143
"is": 1/7 ≈ 0.143
"bright": 1/7 ≈ 0.143

d4:

"we": 1/9 ≈ 0.111
"can": 1/9 ≈ 0.111
"see": 1/9 ≈ 0.111
"the": 2/9 ≈ 0.222
"shining": 1/9 ≈ 0.111
"sun": 2/9 ≈ 0.222
"bright": 1/9 ≈ 0.111

Word Embedding using TF-IDF

Step 3: Calculate Inverse Document Frequency (IDF)

- IDF(t) = log_base 10(Total number of documents / Number of documents with term t)

Term	df	IDF
the	4	**0**
sky	2	**0.301**
is	3	**0.125**
blue	1	**0.602**
sun	3	**0.125**
bright	3	**0.125**
today	1	**0.602**
in	1	**0.602**
we	1	**0.602**
can	1	**0.602**
see	1	**0.602**
shining	1	**0.602**

*d*1 : "The sky is blue.

*d*2 : "The sun is bright today."

*d*3: "The sun in the sky is bright."

*d*4: "We can see the shining sun, the bright sun."

Word Embedding using TF-IDF

Step 4: Calculate TF-IDF

- TF-IDF(t, d) = TF(t, d) * IDF(t)
- TF-IDF Scores:

d1:

Term	TF	IDF	TF-IDF
the	0.25	0	**0**
sky	0.25	0.301	**0.075**
is	0.25	0.125	**0.031**
blue	0.25	0.602	**0.151**

d2:

Term	TF	IDF	TF-IDF
the	0.20	0	**0**
sun	0.20	0.125	**0.025**
is	0.20	0.125	**0.025**
bright	0.20	0.125	**0.025**
today	0.20	0.602	**0.120**

d3:

Term	TF	IDF	TF-IDF
the	0.33	0	**0**
sun	0.17	0.125	**0.021**
in	0.17	0.602	**0.100**
sky	0.17	0.301	**0.050**
is	0.17	0.125	**0.021**
bright	0.17	0.125	**0.021**

d4:

Term	TF	IDF	TF-IDF
we	0.125	0.602	**0.075**
can	0.125	0.602	**0.075**
see	0.125	0.602	**0.075**
shining	0.125	0.602	**0.075**
sun	0.25	0.125	**0.031**
the	0.125	0	**0**
bright	0.125	0.125	**0.016**

Word Embedding using TF-IDF

Final TF-IDF Scores:

- d1{ 0, 0.075,0.031,0.151}
- d2{0, 0.025, 0.025, 0.120}
- d3{0, 0.021,0.100,0.050, 0.021, 0.021}
- D4{ 0.075, 0.075, 0.075, 0.075, 0.031, 0, 0.016}

Word Embedding using TF-IDF

- **Purpose of TF-IDF:**
  TF-IDF is used to find how important a word is in a document compared to other documents.
- **Result of calculation:**
  Common words like *“the”* get very low or zero scores, while unique words like *“blue”*, *“today”*, and *“shining”* get higher scores.
- **Conclusion:**
  The final TF-IDF vectors represent each document using important words, which helps in document comparison and search.

Word2Vec

- Word2Vec (word to vector) is a technique used to convert words to vectors, thereby capturing their meaning, semantic similarity, and relationship with surrounding text.
- This method helps computers learn the context and association/ meaning of expressions and keywords from large text collections such as news articles and books.

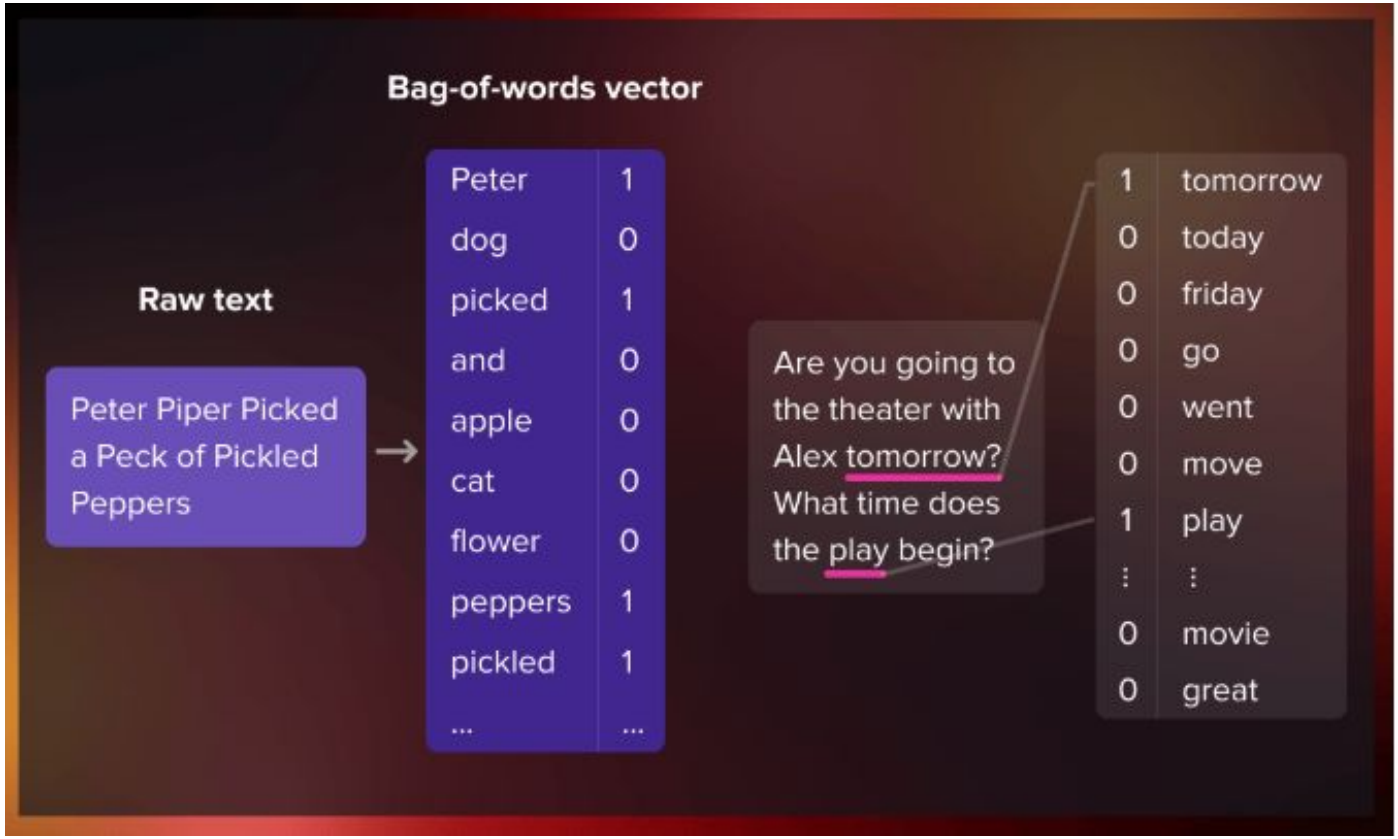
Word2Vec

How It Works

- **Continuous Bag of Words (CBOW):**
  - Predicts a word based on its surrounding context.
- **Skip-Gram:**
  - Predicts surrounding words given a single word.

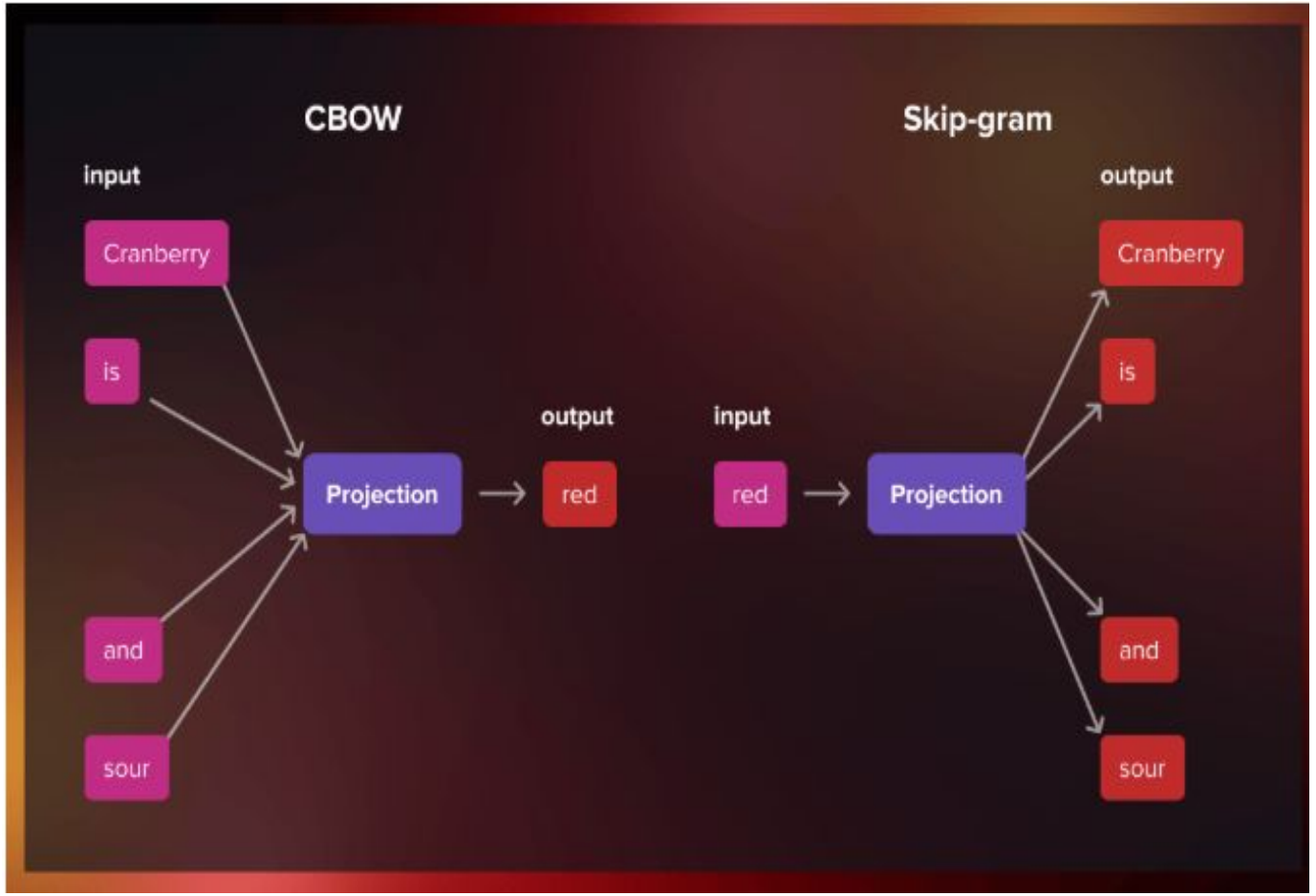
Continuous bag-of-words (CBOW)

- The continuous bag-of-words model predicts the central word using the surrounding context words, which comprises a few words before and after the current word.



Skip-gram

- The skip-gram model architecture is designed to achieve the opposite of the CBOW model.
- Instead of predicting the center word from the surrounding context words, it aims to predict the surrounding context words given the center word.



Explanation Continue

Example:

- Input Sentence: "I love NLP."
- Output:
  - "I" → [0.1, 0.3, 0.5]
  - "love" → [0.2, 0.4, 0.6]
  - "NLP" → [0.3, 0.5, 0.7].

Text Classification with Word Embeddings

Problem:

- Classify movie reviews as "Positive" or "Negative."

Steps:

- **Input:** Reviews like "The movie was great."
- **Preprocessing:** Remove stop words, tokenize, and convert to embeddings.
- **Feature Representation:**
  - Word2Vec generates numerical vectors for words.
- **Model Application:**
  - Train an ML classifier (e.g., Logistic Regression) using embeddings.
- **Output:** Sentiment = Positive.

Visualization:

- Show embeddings in a vector space.

Applications of NLP with Machine Learning

Applications

- **Chatbots and Virtual Assistants**
  - Understand user queries and generate responses.
- **Spam Filtering**
  - Classify emails as spam or not.
- **Sentiment Analysis**
  - Gauge opinions from reviews.
- **Search Engines**
  - Cluster and rank results based on relevance.

Key Takeaway

- Combining NLP and ML makes language processing scalable and intelligent.

Working Example: End-to-End Text Processing with Word2Vec

Step 1: Input and Preprocessing

- **Input Sentence:**
  "NLP is fun"
- **Preprocessing Steps:**
- **Convert to Lowercase:**
  Input → "nlp is fun"
- **Tokenization:**
  Split the sentence into words:
  Tokens → ["nlp", "is", "fun"]
- **Remove Stop Words (if required):**
  Common stop words like "is" can be removed.
  Result → ["nlp", "fun"]

Step 2: Generating Word Embeddings with Word2Vec

- Word2Vec maps each word to a dense numerical vector based on its context in the corpus.
- Assume a pre-trained Word2Vec model generates the following embeddings for each word

Word	Embedding Vector (3D for simplicity)
NLP	[0.5, 0.1, 0.8]
is	[0.2, 0.6, 0.4]
fun	[0.9, 0.7, 0.3]

Vector Representation for the Sentence:

- Sentence Embedding → Combine word embeddings (e.g., by averaging):
- **Sentence Vector** = (NLP+is+fun)/3
- **Sentence Vector** = ([0.5,0.1,0.8]+[0.2,0.6,0.4]+[0.9,0.7,0.3])/3
- **Sentence Vector** = [0.53,0.47,0.5]

Step 3: Feature Extraction

- The sentence embedding $[0.53, 0.47, 0.5]$ is the extracted feature representation for the entire sentence. These features can now be passed into a machine learning model for various tasks, such as classification or clustering.

Step 4: Using a Model

Example Task: Text Classification

- Let's assume the task is to classify the sentence as Positive or Negative.
- Input to Model: Use the sentence embedding [0.53, 0.47, 0.5] as input to a classification model (e.g., Logistic Regression or Neural Network).

Model Training:

- **Training Data:** Use labelled embeddings of sentences (e.g., embeddings of "I love NLP" → Positive, and "I hate this task" → Negative).
- **Train:** The model learns patterns in the embedding space that associate vectors with labels.